

充分運用實驗：透過 Vertex AI 管理機器學習實驗

來源：https://codelabs.developers.google.com/vertex_experiments_pipelines_intro?hl=zh-tw

步驟數：8

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [設定環境](#)
5. [筆記本的初始設定步驟](#)
6. [建立管道](#)
7. [找出成效最佳的執行作業](#)
8. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#) 建構管道，在 TensorFlow 中訓練自訂 Keras 模型。接著，我們會使用 [Vertex AI Experiments](#) 的新功能追蹤及比較模型執行作業，找出能產生最佳效能的超參數組合。

課程內容

內容如下：

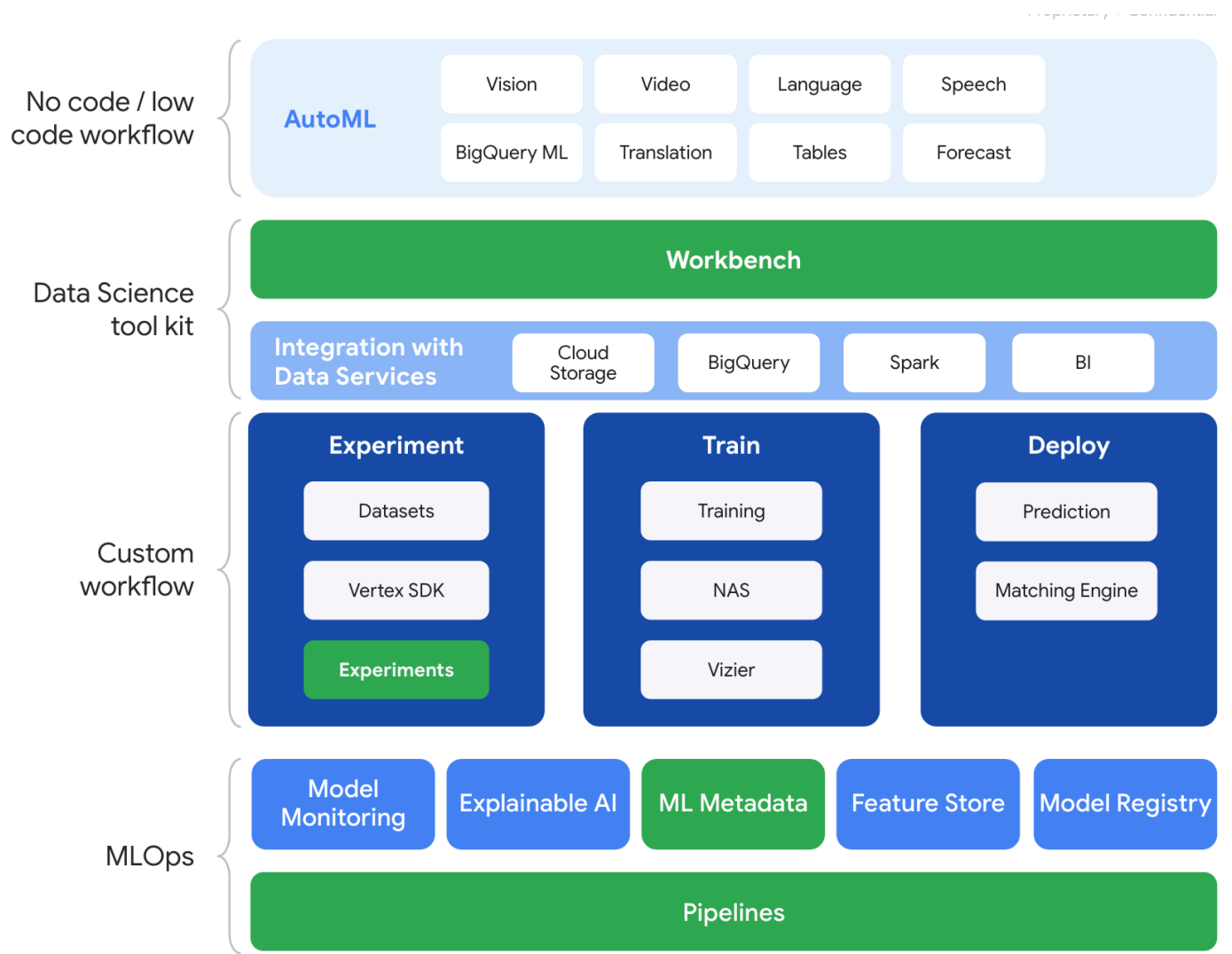
- 訓練自訂 Keras 模型，預測球員評分 (例如迴歸)
- 使用 Kubeflow Pipelines SDK 建構可擴充的機器學習 pipeline
- 建立並執行 5 步驟管道，從 Cloud Storage 擷取資料、調整資料規模、訓練模型、評估模型，然後將產生的模型儲存回 Cloud Storage
- 運用 Vertex ML Metadata 儲存模型構件，例如模型和模型指標
- 使用 Vertex AI Experiments 比較各種管道執行作業的結果

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。Vertex AI 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：實驗、管道、機器學習中繼資料和 Workbench



3. 用途總覽

我們將使用來自 [EA Sports 的 FIFA 電玩遊戲系列](#)，這個資料集非常熱門。包含 2008 年至 2016 年賽季的 25,000 多場足球賽事和 10,000 多名球員。資料已預先處理，方便您輕鬆上手。您會在整個實驗室中使用這個資料集，現在可以在公開 Cloud Storage bucket 中找到。本程式碼研究室稍後會詳細說明如何存取資料集。我們的最終目標是根據各種遊戲內動作 (例如攔截和罰球)，預測球員的整體評分。

為什麼 Vertex AI Experiments 對資料科學很有幫助？

資料科學本質上就是實驗，畢竟他們被稱為科學家。優秀的資料科學家會以假設為導向，透過反覆試驗來測試各種假設，希望透過連續迭代，打造效能更出色的模型。

資料科學團隊雖然已採用實驗，但往往難以追蹤工作，以及透過實驗發現的「獨門秘方」。發生這種情況的原因如下：

- 追蹤訓練工作可能會很麻煩，因此很容易忽略哪些部分成效良好，哪些部分成效不佳
- 如果查看整個資料科學團隊，這個問題會更加嚴重，因為並非所有成員都會追蹤實驗，甚至可能不會與他人分享結果
- 資料擷取作業耗時費力，大多數團隊都採用手動方法 (例如試算表或文件)，導致資訊不一致且不完整，無法從中學習

重點摘要：Vertex AI Experiments 會為您完成工作，協助您更輕鬆地追蹤及比較實驗結果

為什麼遊戲要使用 Vertex AI Experiments？

從歷史來看，遊戲一直是機器學習和 ML 實驗的遊樂場。遊戲每天會產生數十億個即時事件，並運用所有這些資料，透過機器學習和機器學習實驗來提升遊戲內體驗、留住玩家，以及評估平台上的不同玩家。因此我們認為遊戲資料集很適合用於整體實驗練習。

步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。

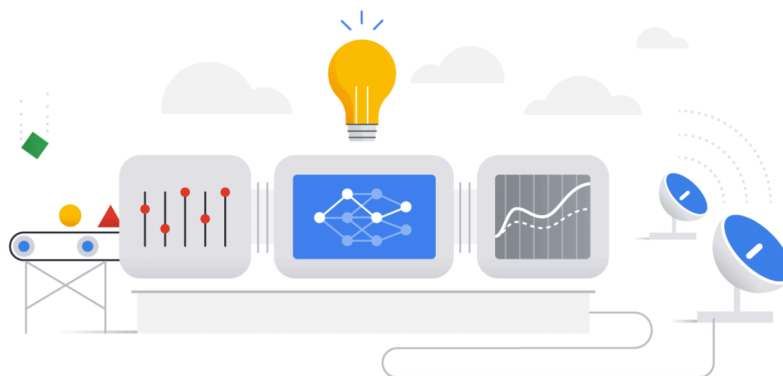
步驟 2：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

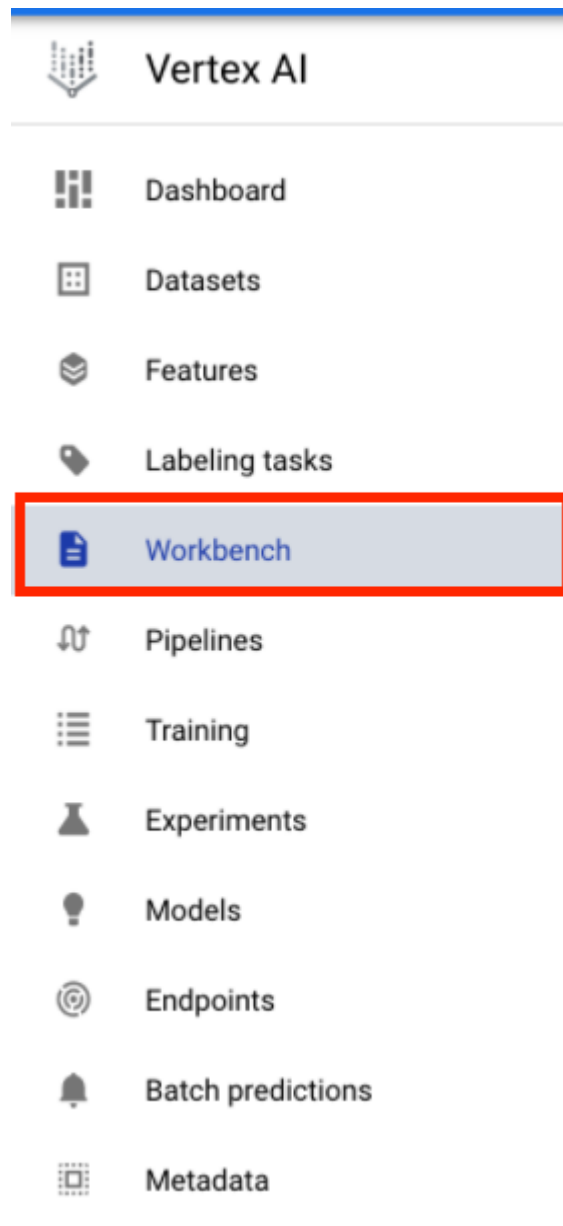
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 3：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：

Managed notebooks

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[■ STOP](#)

[⏻ RESET](#)

[🗑 DELETE](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

EXECUTIONS [PREVIEW](#)

SCHEDULES [PREVIEW](#)

Managed notebooks provide JupyterLab services and flexible computing resources integrated with Google Cloud services. [Learn more](#)

然後選取「新增記事本」。

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

Notebooks service has been moved under the Vertex AI Workbench

為筆記本命名，然後按一下「進階設定」。

Create a managed notebook

Notebook name *

my-notebook

Name must be 63 characters or fewer, must start with a letter and include only lowercase letters, digits or '-'.
Name must be 63 characters or fewer, must start with a letter and include only lowercase letters, digits or '-'.

Region *

us-central1 (Iowa)



Advanced settings



在「進階設定」下方啟用閒置關機功能，並將分鐘數設為 60。也就是說，筆記本會在閒置時自動關機，避免產生不必要的費用。

Idle shutdown

☒ Enable Idle Shutdown 

Time of inactivity before shutdown (Minutes) *

60

Must be integer: 1-600

步驟 4：開啟筆記本

建立執行個體後，請選取「Open JupyterLab」。



my-notebook

OPEN JUPYTERLAB

步驟 5：驗證 (僅限首次)

首次使用新執行個體時，系統會要求您驗證身分。按照 UI 中的步驟操作。

Authenticate your managed notebook

To use managed notebooks with your Google Cloud Platform data, you need to grant Vertex AI Workbench permission to access your data and authenticate your managed notebook. See [Terms of Service](#) and [Privacy Policy](#).

EXIT

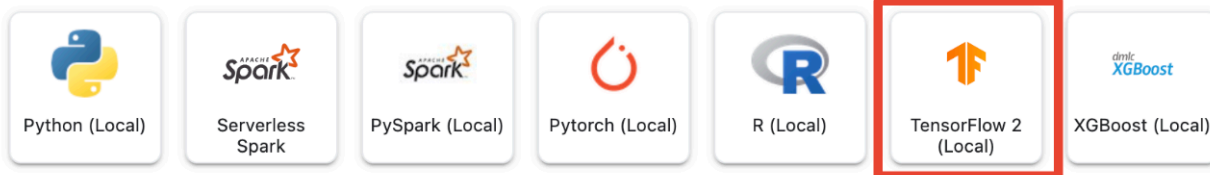
AUTHENTICATE

步驟 6：選取適當的核心

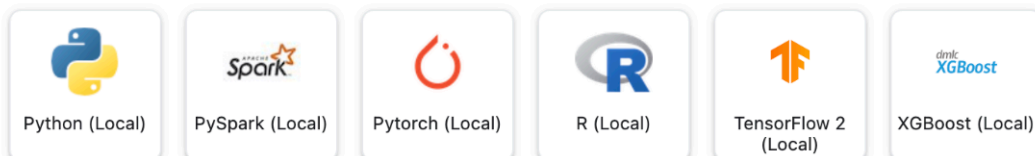
受管理筆記本會在單一 UI 中提供多個核心。選取 Tensorflow 2 (本機) 的核心。



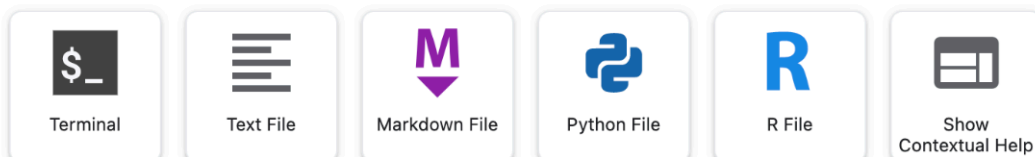
Notebook



Console



Other



步驟 5 · 約 10 分鐘

5. 筆記本的初始設定步驟

您必須先在筆記本中完成一系列額外步驟，設定環境，才能建構管道。這些步驟包括：安裝任何額外套件、設定變數、建立 Cloud Storage bucket、從公開儲存空間 bucket 複製遊戲資料集，以及匯入程式庫和定義額外常數。

步驟 1：安裝其他套件

我們需要安裝筆記本環境目前未安裝的其他套件依附元件。例如 KFP SDK。

```
!pip3 install --user --force-reinstall 'google-cloud-aiplatform>=1.15' -q --no-warn-conflicts
!pip3 install --user kfp -q --no-warn-conflicts
```

接著，請重新啟動 Notebook Kernel，以便在筆記本中使用下載的套件。

```
# Automatically restart kernel after installs
import os

if not os.getenv("IS_TESTING"):
    # Automatically restart kernel after installs
    import IPython

    app = IPython.Application.instance()
    app.kernel.do_shutdown(True)
```

步驟 2：設定變數

我們要定義 `PROJECT_ID`。如果您不知道 `Project_ID`，或許可以使用 `gcloud` 取得 `PROJECT_ID`。

```
import os

PROJECT_ID = ""

# Get your Google Cloud project ID from gcloud
if not os.getenv("IS_TESTING"):
    shell_output = !gcloud config list --format 'value(core.project)' 2>/dev/null
    PROJECT_ID = shell_output[0]
    print("Project ID: ", PROJECT_ID)
```

否則，請在此處設定 `PROJECT_ID`。

```
if PROJECT_ID == "" or PROJECT_ID is None:
    PROJECT_ID = "[your-project-id]" # @param {type:"string"}
```

我們也會想設定 `REGION` 變數，這個變數會用於這個筆記本的其餘部分。以下是 Vertex AI 支援的區域。建議選擇離您最近的區域。

- 美洲：us-central1
- 歐洲：europe-west4

- 亞太地區：asia-east1

請勿使用多區域值區進行 Vertex AI 訓練。並非所有區域都支援所有 Vertex AI 服務。進一步瞭解 [Vertex AI 區域](#)。

```
#set your region
REGION = "us-central1" # @param {type: "string"}
```

最後，我們會設定 `TIMESTAMP` 變數。這個變數可避免使用者在建立資源時發生名稱衝突。您需要為每個執行個體工作階段建立 `TIMESTAMP`，並附加到本教學課程中建立的資源名稱。

```
#set timestamp to avoid collisions between multiple users

from datetime import datetime

TIMESTAMP = datetime.now().strftime("%Y%m%d%H%M%S")
```

步驟 3：建立 Cloud Storage bucket

您需要指定並運用 Cloud Storage 暫存 bucket。暫存值區會保留與資料集和模型資源相關的所有資料，以供跨工作階段使用。

在下方設定 Cloud Storage bucket 名稱。值區名稱在所有 Google Cloud 專案中不得重複，包括您機構以外的專案。

```
#set cloud storage bucket
BUCKET_NAME = "[insert bucket name here]" # @param {type:"string"}
BUCKET_URI = f"gs://{BUCKET_NAME}"
```

如果您的 Cloud Storage bucket 尚不存在，您可以執行下列儲存格來建立 Cloud Storage bucket。

```
! gsutil mb -l $REGION -p $PROJECT_ID $BUCKET_URI
```

接著執行下列儲存格，驗證 Cloud Storage bucket 的存取權。

```
#verify access
! gsutil ls -al $BUCKET_URI
```

步驟 4：複製我們的遊戲資料集

如前所述，您將運用 EA Sports 熱門電玩遊戲 FIFA 的熱門遊戲資料集。我們已為您完成前置處理工作，您只需要從公開儲存空間值區複製資料集，然後移至您建立的值區即可。

```
# copy the data over to your cloud storage bucket
DATASET_URI = "gs://cloud-samples-data/vertex-ai/structured_data/player_data"

!gsutil cp -r $DATASET_URI $BUCKET_URI
```

步驟 5：匯入程式庫並定義其他常數

接著，我們要匯入 Vertex AI、KFP 等程式庫。

```
import logging
import os
import time

logger = logging.getLogger("logger")
logging.basicConfig(level=logging.INFO)

import kfp.v2.compiler as compiler
# Pipeline Experiments
import kfp.v2.dsl as dsl
# Vertex AI
from google.cloud import aiplatform as vertex_ai
from kfp.v2.dsl import Artifact, Input, Metrics, Model, Output, component
from typing import NamedTuple
```

我們也會定義其他常數，以便在筆記本的其餘部分中參照，例如訓練資料的檔案路徑。

```
#import libraries and define constants
# Experiments

TASK = "regression"
MODEL_TYPE = "tensorflow"
EXPERIMENT_NAME = f"{PROJECT_ID}-{TASK}-{MODEL_TYPE}-{TIMESTAMP}"

# Pipeline
PIPELINE_URI = f"{BUCKET_URI}/pipelines"
TRAIN_URI = f"{BUCKET_URI}/player_data/data.csv"
LABEL_URI = f"{BUCKET_URI}/player_data/labels.csv"
MODEL_URI = f"{BUCKET_URI}/model"
DISPLAY_NAME = "experiments-demo-gaming-data"
BQ_DATASET = "player_data"
BQ_LOCATION = "US"
VIEW_NAME = 'dataset_test'
PIPELINE_JSON_PKG_PATH = "experiments_demo_gaming_data.json"
PIPELINE_ROOT = f"gs://{BUCKET_URI}/pipeline_root"
```

步驟 6 · 約 0 分鐘

6. 建立管道

現在可以開始運用 Vertex AI 建構訓練 pipeline。我們會初始化 Vertex AI SDK、將訓練工作設為 pipeline 元件、建構 pipeline、提交 pipeline 執行作業，並運用 Vertex AI SDK 查看實驗及監控狀態。

步驟 1：初始化 Vertex AI SDK

初始化 Vertex AI SDK，並設定 `PROJECT_ID` 和 `BUCKET_URI`。

```
#initialize vertex AI SDK
vertex_ai.init(project=PROJECT_ID, staging_bucket=BUCKET_URI)
```

步驟 2：將訓練工作設為 Pipeline 元件

如要開始執行實驗，我們需要將訓練工作定義為管道元件，藉此指定訓練工作。我們的管道會將訓練資料和超參數 (例如 `DROPOUT_RATE`、`LEARNING_RATE`、`EPOCHS`) 做為輸入內容，並輸出模型指標 (例如 `MAE` 和 `RMSE`) 和模型構件。

想瞭解如何搭配其他類型的模型，在管道以外使用 Vertex AI Experiments 嗎？請參閱這個範例，瞭解如何試用在本機訓練的模型 ([請參閱比較在本機訓練的模型範例](#))。

```

@Component(
    packages_to_install=[
        "numpy==1.21.0",
        "pandas==1.3.5",
        "scikit-learn==1.0.2",
        "tensorflow==2.9.0",
    ]
)

def custom_trainer(
    train_uri: str,
    label_uri: str,
    dropout_rate: float,
    learning_rate: float,
    epochs: int,
    model_uri: str,
    metrics: Output[Metrics],
    model_metadata: Output[Model],

):

    # import libraries
    import logging
    import uuid
    from pathlib import Path as path

    import pandas as pd
    import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense
    from tensorflow.keras.layers import Dropout
    from tensorflow.keras.metrics import Metric
    from sklearn.metrics import accuracy_score
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.metrics import mean_absolute_error
    import numpy as np
    from math import sqrt
    import os
    import tempfile

    # set variables and use gcsfuse to update prefixes
    gs_prefix = "gs://"
    gcsfuse_prefix = "/gcs/"
    train_path = train_uri.replace(gs_prefix, gcsfuse_prefix)
    label_path = label_uri.replace(gs_prefix, gcsfuse_prefix)
    model_path = model_uri.replace(gs_prefix, gcsfuse_prefix)

    def get_logger():

        logger = logging.getLogger(__name__)

```

```

logger.setLevel(logging.INFO)
handler = logging.StreamHandler()
handler.setFormatter(
    logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
)
logger.addHandler(handler)
return logger

def get_data(
    train_path: str,
    label_path: str
) -> (pd.DataFrame):

    #load data into pandas dataframe
    data_0 = pd.read_csv(train_path)
    labels_0 = pd.read_csv(label_path)

    #drop unnecessary leading columns

    data = data_0.drop('Unnamed: 0', axis=1)
    labels = labels_0.drop('Unnamed: 0', axis=1)

    #save as numpy array for reshaping of data

    labels = labels.values
    data = data.values

    # Split the data
    labels = labels.reshape((labels.size,))
    train_data, test_data, train_labels, test_labels = train_test_split(data, labels,
test_size=0.2, shuffle=True, random_state=7)

    #Convert data back to pandas dataframe for scaling

    train_data = pd.DataFrame(train_data)
    test_data = pd.DataFrame(test_data)
    train_labels = pd.DataFrame(train_labels)
    test_labels = pd.DataFrame(test_labels)

    #Scale and normalize the training dataset

    scaler = StandardScaler()
    scaler.fit(train_data)
    train_data = pd.DataFrame(scaler.transform(train_data), index=train_data.index,
columns=train_data.columns)
    test_data = pd.DataFrame(scaler.transform(test_data), index=test_data.index,
columns=test_data.columns)

    return train_data, train_labels, test_data, test_labels

""" Train your Keras model passing in the training data and values for learning rate,

```


dropout rate,and the number of epochs """

```
def train_model(
    learning_rate: float,
    dropout_rate: float,
    epochs: float,
    train_data: pd.DataFrame,
    train_labels: pd.DataFrame):

    # Train tensorflow model
    param = {"learning_rate": learning_rate, "dropout_rate": dropout_rate, "epochs":
epochs}

    model = Sequential()
    model.add(Dense(500, input_dim=train_data.shape[1], activation= "relu"))
    model.add(Dropout(param['dropout_rate']))
    model.add(Dense(100, activation= "relu"))
    model.add(Dense(50, activation= "relu"))
    model.add(Dense(1))

    model.compile(
        tf.keras.optimizers.Adam(learning_rate= param['learning_rate']),
        loss='mse',
        metrics=
[tf.keras.metrics.RootMeanSquaredError(),tf.keras.metrics.MeanAbsoluteError()])

    model.fit(train_data, train_labels, epochs= param['epochs'])

    return model

# Get Predictions
def get_predictions(model, test_data):

    dtest = pd.DataFrame(test_data)
    pred = model.predict(dtest)
    return pred

# Evaluate predictions with MAE
def evaluate_model_mae(pred, test_labels):

    mae = mean_absolute_error(test_labels, pred)
    return mae

# Evaluate predictions with RMSE
def evaluate_model_rmse(pred, test_labels):

    rmse = np.sqrt(np.mean((test_labels - pred)**2))
    return rmse

#Save your trained model in GCS
def save_model(model, model_path):
```

```

model_id = str(uuid.uuid1())
model_path = f"{model_path}/{model_id}"
path(model_path).parent.mkdir(parents=True, exist_ok=True)
model.save(model_path + '/model_tensorflow')

# Main -----

train_data, train_labels, test_data, test_labels = get_data(train_path, label_path)
model = train_model(learning_rate, dropout_rate, epochs, train_data, train_labels )
pred = get_predictions(model, test_data)
mae = evaluate_model_mae(pred, test_labels)
rmse = evaluate_model_rmse(pred, test_labels)
save_model(model, model_path)

# Metadata -----

#convert numpy array to pandas series
mae = pd.Series(mae)
rmse = pd.Series(rmse)

#log metrics and model artifacts with ML Metadata. Save metrics as a list.
metrics.log_metric("mae", mae.to_list())
metrics.log_metric("rmse", rmse.to_list())
model_metadata.uri = model_uri

```

步驟 3：建構管道

現在我們要使用 KFP 提供的 `Domain Specific Language (DSL)` 設定工作流程，並將管道編譯為 `JSON` 檔案。

```

# define our workflow

@dsl.pipeline(name="gaming-custom-training-pipeline")
def pipeline(
    train_uri: str,
    label_uri: str,
    dropout_rate: float,
    learning_rate: float,
    epochs: int,
    model_uri: str,
):

    custom_trainer(
        train_uri, label_uri, dropout_rate, learning_rate, epochs, model_uri
    )

```

```

#compile our pipeline
compiler.Compiler().compile(pipeline_func=pipeline, package_path="gaming_pipeline.json")

```

步驟 4：提交 Pipeline Run

我們已完成設定元件和定義管道的艱鉅工作。我們已準備好提交上述管道的各種執行作業。為此，我們需要定義不同超參數的值，如下所示：

```
runs = [
    {"dropout_rate": 0.001, "learning_rate": 0.001, "epochs": 20},
    {"dropout_rate": 0.002, "learning_rate": 0.002, "epochs": 25},
    {"dropout_rate": 0.003, "learning_rate": 0.003, "epochs": 30},
    {"dropout_rate": 0.004, "learning_rate": 0.004, "epochs": 35},
    {"dropout_rate": 0.005, "learning_rate": 0.005, "epochs": 40},
]
```

定義超參數後，我們就能運用 `for loop` 順利饋送管道的不同執行項目：

```
for i, run in enumerate(runs):

    job = vertex_ai.PipelineJob(
        display_name=f"{EXPERIMENT_NAME}-pipeline-run-{i}",
        template_path="gaming_pipeline.json",
        pipeline_root=PIPELINE_URI,
        parameter_values={
            "train_uri": TRAIN_URI,
            "label_uri": LABEL_URI,
            "model_uri": MODEL_URI,
            **run,
        },
    )
    job.submit(experiment=EXPERIMENT_NAME)
```

步驟 5：運用 Vertex AI SDK 查看實驗

您可以使用 Vertex AI SDK 監控 pipeline 執行作業的狀態。您也可以使用這項功能，傳回 Vertex AI 實驗中 Pipeline Run 的參數和指標。使用下列程式碼，查看與執行作業相關聯的參數及其目前狀態。

```
# see state/status of all the pipeline runs

vertex_ai.get_experiment_df(EXPERIMENT_NAME)
```

Python 適用的 Vertex AI SDK 目前支援 `get_experiment_df` 方法，可協助您傳回執行作業的詳細資料。您可以搭配其他 `PipelineJob` 方法，擷取各管道工作的詳細資料。提交後，執行時間應該只需要約 5 分鐘。

您可以運用下列程式碼，取得管道執行狀態的最新資訊。

```
#check on current status
while True:
    pipeline_experiments_df = vertex_ai.get_experiment_df(EXPERIMENT_NAME)
    if all(
        pipeline_state != "COMPLETE" for pipeline_state in pipeline_experiments_df.state
    ):
        print("Pipeline runs are still running...")
        if any(
            pipeline_state == "FAILED"
            for pipeline_state in pipeline_experiments_df.state
        ):
            print("At least one Pipeline run failed")
            break
    else:
        print("Pipeline experiment runs have completed")
        break
    time.sleep(60)
```

您也可以使用 `run_name` 呼叫特定管道作業。

```
# Call the pipeline runs based on the experiment run name
pipeline_experiments_df = vertex_ai.get_experiment_df(EXPERIMENT_NAME)
job = vertex_ai.PipelineJob.get(pipeline_experiments_df.run_name[0])
print(job.resource_name)
print(job._dashboard_uri())
```

最後，您可以按照設定的時間間隔 (例如每 60 秒) 重新整理執行作業的狀態，查看狀態從 `RUNNING` 變更為 `FAILED` 或 `COMPLETE`。

```
# wait 60 seconds and view state again
import time
time.sleep(60)
vertex_ai.get_experiment_df(EXPERIMENT_NAME)
```

7. 找出成效最佳的執行作業

太棒了，我們現在有管道執行的結果。你可能會想知道，這些結果能提供哪些資訊？實驗輸出內容應包含五列，管道每次執行都會產生一行。畫面看起來大致如下：

state	param.epochs	param.dropout_rate	param.model_uri	param.label_uri	param.train_uri	param.learning_rate	metric.rmse	metric.mae
COMPLETE	40	0.005	gs://experiments-demo-gaming-data/model	gs://experiments-demo-gaming-data/player-data/...	gs://experiments-demo-gaming-data/player-data/...	0.005	[1.1994915338433996]	[0.9282052047070797]
COMPLETE	35	0.004	gs://experiments-demo-gaming-data/model	gs://experiments-demo-gaming-data/player-data/...	gs://experiments-demo-gaming-data/player-data/...	0.004	[1.6774447216293535]	[1.3912596980966023]
COMPLETE	30	0.003	gs://experiments-demo-gaming-data/model	gs://experiments-demo-gaming-data/player-data/...	gs://experiments-demo-gaming-data/player-data/...	0.003	[1.368606749212623]	[1.0496017500678982]
COMPLETE	25	0.002	gs://experiments-demo-gaming-data/model	gs://experiments-demo-gaming-data/player-data/...	gs://experiments-demo-gaming-data/player-data/...	0.002	[1.7340842296667147]	[1.3291234783216863]
COMPLETE	20	0.001	gs://experiments-demo-gaming-data/model	gs://experiments-demo-gaming-data/player-data/...	gs://experiments-demo-gaming-data/player-data/...	0.001	[1.0703339726645948]	[0.8067645053795942]

MAE 和 RMSE 都是模型預測平均誤差的測量指標，因此在大多數情況下，這兩項指標的值越低越好。從 Vertex AI Experiments 的輸出內容中，我們可以看到最後一次執行在兩個指標中都最成功，`dropout_rate` 為 0.001，`learning_rate` 為 0.001，`epochs` 總數為 20。根據這項實驗，這些模型參數最終會用於正式環境，因為這樣可獲得最佳模型效能。

這樣就完成了實驗室！



您已瞭解如何使用 Vertex AI 執行下列作業：

- 訓練自訂 Keras 模型，預測球員評分 (例如迴歸)
- 使用 Kubeflow Pipelines SDK 建構可擴充的機器學習 pipeline
- 建立並執行 5 步驟管道，從 GCS 擷取資料、調整資料大小、訓練模型、評估模型，然後將產生的模型存回 GCS
- 運用 Vertex ML Metadata 儲存模型構件，例如模型和模型指標
- 使用 Vertex AI Experiments 比較各種管道執行作業的結果

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 8 · 約 1 分鐘

8. 清除所用資源

為避免產生費用，建議您刪除在本實驗室中建立的資源。

步驟 1：停止或刪除 Notebooks 執行個體

如要繼續使用在本實驗室中建立的筆記本，建議您在未使用時關閉筆記本。在 Cloud 控制台的 Notebooks 使用者介面中，選取筆記本，然後選取「停止」。如要完全刪除執行個體，請選取「刪除」：

Notebooks  NEW INSTANCE  REFRESH  START  STOP  RESET  DELETE

Create and use Jupyter Notebooks with a notebook instance. Notebook instances have JupyterLab pre-installed and are configured with GPU-enabled machine learning frameworks. [Learn more](#)

 Filter Enter property name or value

		Instance name 		Zone	Environment version	Auto-upgrade
		tensorflow-2-3-20210405-151653	OPEN JUPYTERLAB	us-west1-b	M65	—

步驟 2：刪除 Cloud Storage bucket

如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

Browser  CREATE BUCKET  DELETE  REFRESH

 Filter Filter buckets

	Name 	Created
	my-awesome-bucket	5 Apr 2021, 15:32:48